Under What Conditions Does Episodic Memory Improve LLM Agent Performance? A Two-Environment Comparative Study

Anonymous ACL submission

Abstract

LLM-based agents are increasingly augmented with episodic memory systems that store and retrieve past experiences to guide future decisions. However, it remains unclear *under what conditions* such memory actually improves performance. We investigate this question through a controlled comparative study across two interactive environments: ALFWorld, a structured household task environment with compositional goals and a fixed action grammar, and WebShop, an open-ended e-commerce environment with high product cardinality and free-form navigation. We propose a memory-centric framework in which agents store outcome-validated episodic experiences—explicitly associating encountered issues with successful resolutions and their observed downstream outcomes—and retrieve relevant past cases via two complementary mechanisms: *Context Retrieval* (CR), which proactively supplies relevant learnings at task start based on semantic similarity, and *Tool Retrieval* (TR), an on-demand hybrid dense-sparse retrieval system the agent can invoke when stuck. On ALFWorld, our memory system substantially improves over the ReAct baseline: CR alone yields +15.7 percentage points (pp) on seen tasks and +14.2pp on unseen tasks, while CR+TR achieves +16.4pp/+19.4pp—approaching Reflexion's accuracy (+20.7pp/+21.6pp) at roughly one-third the step cost. On WebShop, the same retrieval mechanisms *degrade* performance by 17–31pp, with even the baseline ReAct agent outperforming all memory-augmented variants. A hard-negative ablation that retrieves maximally *irrelevant* experiences reveals a striking paradox: in WebShop, irrelevant memories (−12.5pp) cause less degradation than semantically "relevant" ones (−30.5pp), suggesting that the failure mode is not noise injection but active misdirection by superficially similar but non-transferable experiences. We analyze this divergence through the lens of three environment properties: *task compositionality* (templated vs. open-ended goals), *action space regularity* (fixed verbs vs. free-form navigation), and *learning transfer radius* (how many tasks benefit from a single learned experience). Our results provide a diagnostic framework and actionable guidance for practitioners on when episodic memory augmentation is—and is not—beneficial for LLM agents.

1 Introduction

LLM-based agents are increasingly augmented with episodic memory—systems that store past experiences and retrieve them to guide future decisions. Recent work has demonstrated impressive gains from such augmentation: Voyager (Wang et al., 2023) builds a skill library of verified programs in Minecraft, GITM (Zhu et al., 2023) maintains a memory of successful goal decompositions, MemoryBank (Zhong et al., 2024) equips conversational agents with long-term memory governed by forgetting curves, and Generative Agents (Park et al., 2023) use a memory stream with reflection to simulate believable human behavior. A common thread runs through these systems: the implicit assumption that storing and retrieving past experience will improve agent performance. But does it always?

We investigate this question through a controlled comparative study across two interactive environments: ALFWorld (Shridhar et al., 2021), a structured household task environment with compositional goals and a fixed action grammar, and WebShop (Yao et al., 2022), an open-ended e-commerce environment with high product cardinality and free-form navigation. The answer, it turns out, depends critically on properties of the environment. On ALFWorld, our memory system improves over the ReAct (Yao et al., 2023) baseline by up to +19.4 percentage points on unseen tasks—nearly matching Reflexion (Shinn et al., 2023) (+21.6pp) at roughly one-third the step cost. On WebShop, the *same* memory system *degrades* performance by up to 30.5pp. Strikingly, a hard-negative control that retrieves maximally *irrelevant* experiences (−12.5pp) outperforms the properly configured system (−30.5pp), revealing that semantically similar but non-transferable memories are more harmful than obviously irrelevant ones. This ∼47pp swing in the effect of the same retrieval system across two environments—consistent across both development and held-out splits—is the central empirical finding of this paper.

Our memory-centric framework stores *outcome-validated episodic experiences*: structured records that explicitly associate encountered issues with successful resolutions and their observed downstream outcomes. Agents retrieve these experiences through two complementary mechanisms. *Context Retrieval* (CR) proactively supplies relevant learnings at task start by embedding the new task description and retrieving the most similar historical experiences. *Tool Retrieval* (TR) provides an on-demand hybrid dense-sparse retrieval tool that the agent can invoke mid-execution when it encounters difficulty. Crucially, we apply the identical retrieval system, the same base LLM, and the same knowledge-base construction pipeline to both environments, ensuring that the observed divergence in outcomes is attributable to environment properties rather than implementation choices.

Our analysis traces this divergence to three environment properties that together predict whether episodic memory will be effective. First, *task compositionality*: ALFWorld tasks decompose into shared sub-goal phases (search, acquire, transform, place) that recur across tasks, whereas WebShop tasks involve unique product searches with little structural overlap. Second, *action space regularity*: ALFWorld's fixed grammar of ∼15 action templates means that learned action sequences (e.g., "open a container before taking an object") generalize broadly, while WebShop's free-form search queries and arbitrary clickable elements yield an effectively infinite action space where specific navigation paths rarely transfer. Third, *learning transfer radius*: in ALFWorld, a single learned experience can benefit dozens of future tasks across multiple task types; in WebShop, each learning is tied to a unique product, yielding a transfer radius near zero. When all three properties favor transfer, memory provides substantial gains; when they do not, retrieved experiences actively mislead the agent.

We argue that this characterization of *when memory fails* constitutes a stronger contribution than a system that claims universal improvement. Rather than presenting yet another memory-augmented agent, we provide practitioners with a diagnostic framework for deciding whether episodic memory augmentation is appropriate for their deployment setting. In summary, our contributions are:

1. A **memory-centric agent framework** featuring outcome-validated episodic memory with two retrieval mechanisms—proactive Context Retrieval (CR) and on-demand hybrid Tool Retrieval (TR)—evaluated under controlled conditions across two diverse environments.

2. A **controlled empirical study** demonstrating that the same memory system produces opposite effects depending on environment properties: up to +19.4pp improvement on ALFWorld versus up to −30.5pp degradation on WebShop, with hard-negative ablations revealing that irrelevant memories cause less harm than semantically "relevant" but non-transferable ones in low-compositionality environments.

3. A **diagnostic framework** based on task compositionality, action space regularity, and learning transfer radius that predicts when episodic memory will help or hurt, providing actionable guidance for practitioners.

2 Related Work

Tool-augmented reasoning agents. A growing body of work equips large language models with explicit reasoning traces and tool-use capabilities. ReAct (Yao et al., 2023) interleaves chain-of-thought reasoning with environment actions in a thought–action–observation loop, enabling grounded decision-making in interactive settings. Reflexion (Shinn et al., 2023) extends this paradigm by adding a critique-and-retry mechanism: after a failed episode, the agent generates a natural-language reflection that is prepended to subsequent attempts, yielding substantial gains on ALFWorld and HotPotQA. Self-Refine (Madaan et al., 2023) applies iterative self-feedback within a single generation pass. Our agent architecture builds on the ReAct backbone augmented with Reflexion-style retries; our contribution is not a new reasoning strategy but rather a systematic investigation of *when* layering episodic memory on top of such agents is beneficial.

Memory-augmented LLM agents. Several recent systems augment LLM agents with persistent memory stores. Voyager (Wang et al., 2023) maintains a skill library of verified JavaScript programs for open-ended Minecraft exploration, retrieving and composing previously learned skills to solve novel tasks. GITM (Zhu et al., 2023) similarly targets Minecraft, decomposing goals into sub-goals and storing successful plans in a reference memory for future retrieval. Generative Agents (Park et al., 2023) introduce a memory stream architecture in which observations are stored, retrieved by recency, relevance, and importance, and periodically consolidated through reflection. MemoryBank (Zhong et al., 2024) incorporates Ebbinghaus forgetting curves to manage long-term memory in conversational settings. A common thread across these works is the implicit assumption that memory accumulation is net-positive: more experience stored means better future performance. Our work challenges this assumption by demonstrating that the same episodic memory mechanism that yields 15–19 percentage-point gains in one environment can *degrade* performance by 17–31 percentage points in another, depending on structural properties of the task distribution.

Interactive decision-making benchmarks. We evaluate on two complementary benchmarks that span a broad spectrum of task structure. ALFWorld (Shridhar et al., 2021) provides a text-based interface to the ALFRED embodied household benchmark, offering six compositional task types (e.g., pick-and-place, heat-then-place) with a fixed action grammar over a closed set of objects and receptacles. Tasks share substantial sub-goal structure—the sequence *locate* $\rightarrow$ *acquire* $\rightarrow$ *transform* $\rightarrow$ *place* recurs across task types—making past experience highly reusable. WebShop (Yao et al., 2022) simulates e-commerce product search and purchase; the agent must issue free-form search queries and navigate product pages to find items matching detailed natural-language specifications. Despite a seemingly simple action interface (`search[query]`, `click[element]`), the effective action space is enormous: each task involves a unique product with unique attributes, and successful strategies rarely transfer across products. This structural contrast—compositional and regular versus open-ended and idiosyncratic—motivates our comparative design and underpins the divergent memory effects we observe.

Retrieval-augmented generation. Retrieval-augmented generation (RAG) (Lewis et al., 2020) improves language model outputs by conditioning on documents retrieved from an external corpus, and has become a standard technique for knowledge-intensive NLP tasks. Subsequent work has extended RAG to agentic settings, where retrieved items are past trajectories or tool-use demonstrations rather than static documents. Our episodic memory mechanisms—context retrieval and tool retrieval—can be viewed as instances of experience-augmented generation. However, while the RAG literature largely focuses on improving retrieval quality under the assumption that relevant retrieval is beneficial, our results reveal a complementary failure mode: when the retrieved experiences are *superficially* similar but *structurally* non-transferable, retrieval actively misleads the agent. This finding highlights the importance of characterizing the *transfer radius* of retrieved experiences, a dimension largely absent from current RAG evaluations.

Positioning. The works above each propose a memory or retrieval mechanism and evaluate it in settings where it succeeds. Our contribution is orthogonal: rather than introducing a new memory architecture, we conduct a controlled comparative study that yields both a strong positive result (ALFWorld) and a principled negative result (WebShop) for the *same* memory mechanisms. We distill these findings into a diagnostic framework based on three task properties—compositionality, action-space regularity, and learning transfer radius—that practitioners can use to predict whether episodic memory will help or hurt in a new domain. To our knowledge, this is the first work to systematically characterize the conditions under which episodic memory improves LLM agent performance, providing actionable guidance beyond the prevailing assumption that more memory is always better.

3 Methodology

We investigate whether episodic memory improves LLM agent performance by evaluating a unified memory-centric framework across two interactive environments with contrasting structural properties. This section describes the environments, the base agent, the memory retrieval mechanisms, and the baselines.

3.1 Environments

ALFWorld. ALFWorld (Shridhar et al., 2021) provides a text-based interface to the ALFRED embodied household benchmark. Agents receive natural-language observations and issue actions from a fixed grammar (e.g., `go to`, `take`, `put`, `open`, `heat`, `cool`, `clean`). The environment defines six task types—*pick_and_place*, *pick_clean_then_place*, *pick_heat_then_place*, *pick_cool_then_place*, *look_at_obj_in_light*, and *pick_two_obj_and_place*—each following a compositional goal structure decomposable into phases: Search → Acquire → Transform → Place. The object vocabulary is small (∼50 object types, ∼20 receptacle types), and the action space is constrained to approximately 15 verb templates. These properties yield high structural overlap between tasks: a learning acquired during one *pick_heat_then_place* task (e.g., "open the microwave before taking the heated object") transfers directly to other tasks of the same type.

WebShop. WebShop (Yao et al., 2022) is an e-commerce web navigation environment in which the agent must find and purchase a product matching a natural-language specification (e.g., "Buy a red cotton shirt under $30 with free shipping"). The action space consists of `search[query]` and `click[element]`, which are syntactically simple but combinatorially vast: the agent must compose free-form search queries and navigate dynamically generated product pages. Each task targets a unique product with unique attributes, resulting in minimal structural overlap between tasks. Unlike ALFWorld, there is no shared sub-task decomposition—each product search traces a distinct path through the catalog.

3.2 Base Agent: ReAct

All agents in our study build on the ReAct framework (Yao et al., 2023), which interleaves *reasoning traces* (thoughts) with *actions* in a loop: at each step, the agent generates a thought analyzing the current observation, selects an action, and receives an environment observation. We use Gemini 2.5 Flash as the base LLM for all experiments. The ReAct agent executes a single trajectory per task with no retries, serving as the baseline for all comparisons.

3.3 Memory Systems

Our memory framework augments the ReAct agent with a persistent knowledge base of past experiences and two complementary retrieval mechanisms: *Context Retrieval* (CR), which proactively injects relevant learnings at the start of each task, and *Tool Retrieval* (TR), an on-demand mechanism the agent can invoke when it encounters an obstacle during execution.

3.3.1 Knowledge Base Construction

The knowledge base is constructed offline from training trajectories using LLM-based extraction (Gemini). For each task across multiple trials, we extract *issue–learning pairs*: structured records that associate a specific mistake (the *issue*) with its successful correction (the *learning*). Each entry contains the following fields:

- **Task metadata**: task description, object type, action verbs, goal phase (Search, Acquire, Transform, Place, or Recover).

- **Issue–learning pair**: `issue_text` (the mistake) and `learning_text` (the correction).
- **Validation level**: a confidence label reflecting downstream outcome:
  - VALID_NEXT_TRIAL: the learning was applied and led to success in a subsequent trial (highest confidence).
  - VALID_SAME_TRIAL: the learning is associated with success within the same trial.
  - CANDIDATE: the learning was generated but the task was not completed (lowest confidence).
- **Provenance**: references to the source task, trial number, and step range for both the issue and its resolution.

We assign scalar validation weights ω to each level: $\omega_{\text{next}} = 1.0$, $\omega_{\text{same}} = 0.9$, and $\omega_{\text{cand}} = 0.6$. These weights are used by both retrieval mechanisms to prioritize high-confidence learnings.

3.3.2 Context Retrieval (CR)

Context Retrieval operates proactively: before the agent takes any action, CR retrieves relevant past learnings based on semantic similarity between the new task description and historical tasks. The retrieved learnings are injected into the agent's prompt as structured guidance.

Embedding and similarity search. We embed all task descriptions using Google's `text-embedding-004` model. Given a new task description q, we compute the cosine similarity between its embedding $\mathbf{v}_q$ and the embeddings of all unique historical task descriptions in the knowledge base:

$$\text{sim}(\mathbf{v}_q, \mathbf{v}_t) = \frac{\mathbf{v}_q \cdot \mathbf{v}_t}{\|\mathbf{v}_q\| \, \|\mathbf{v}_t\|} \tag{1}$$

We retrieve the top-k (k=5) most similar tasks from a pre-built learning counts index, which tracks distinct task descriptions and the number of associated learnings. Exact matches (similarity ≥ 1.0) are filtered to prevent data leakage.

Dynamic context sizing. Rather than retrieving a fixed number of learnings, CR adapts the context window to the complexity of the neighborhood. Let $L_{\max}$ denote the maximum learning count among the top-k tasks. The retrieval quota is:

$$N_{\text{pick}} = \min(\lceil \alpha \cdot L_{\max} \rceil \,,\ N_{\max}) \tag{2}$$

where $\alpha = 1.5$ is a scaling factor and $N_{\max} = 25$ is a hard cap. This ensures the agent receives enough context to cover the most learning-rich neighbor while avoiding prompt overflow.

Scoring and re-ranking. All learnings associated with the top-k tasks are pooled and scored using a multiplicative formula that combines semantic relevance with validation confidence:

$$\text{Score}_{\text{CR}}(\ell) = \text{sim}(\mathbf{v}_q, \mathbf{v}_{t(\ell)}) \times \omega_\ell \tag{3}$$

where $t(\ell)$ is the source task of learning ℓ and ω_ℓ is its validation weight. Learnings are sorted by this score in descending order, deduplicated, and the top N_{pick} are selected.

Prompt construction. The selected learnings are formatted as numbered issue–learning pairs and prepended to the agent's task prompt, providing explicit guidance on mistakes to avoid and their corrections.

The full procedure is given in Algorithm 1.

3.3.3 Tool Retrieval (TR)

Tool Retrieval provides an on-demand retrieval mechanism exposed as an action the agent can invoke during execution. When the agent encounters a specific obstacle (e.g., "cannot find the knife"), it calls `help["issue description"]`, triggering a hybrid dense–sparse retrieval over the knowledge base's issue–learning entries. Unlike CR, which operates on task-level similarity, TR matches at the issue level, targeting the agent's immediate problem.

Query preprocessing. The agent's issue description is cleaned by removing digits (to generalize across instance IDs) and normalizing whitespace.

Hybrid retrieval. TR combines two complementary retrieval strategies:

- **Dense retrieval**: The cleaned query is embedded using `text-embedding-004`, and cosine similarity is computed against pre-cached embeddings of all `issue_text` fields in the knowledge base. This captures semantic similarity.

Algorithm 1 Context Retrieval from Knowledge Base

Require: Task description q; knowledge base $\mathcal{K}$; neighbor count k=5; scaling factor α=1.5; cap $N_{\max}$=25; validation weights W

Ensure: Selected learnings C

$\mathbf{v}_q \leftarrow \textsc{Embed}(q)$
$\mathcal{T}_{\text{top}} \leftarrow$ Top-k tasks in $\mathcal{K}$ by $\text{sim}(\mathbf{v}_q, \mathbf{v}_t)$ {filter exact matches}
$L_{\max} \leftarrow \max_{t \in \mathcal{T}_{\text{top}}} \textsc{LearningCount}(t)$
$N_{\text{pick}} \leftarrow \min(\lceil \alpha \cdot L_{\max} \rceil,\ N_{\max})$
$\mathcal{L}_{\text{pool}} \leftarrow \emptyset$
for each $(t, s_t) \in \mathcal{T}_{\text{top}}$ **do**
  for each learning $\ell \in \textsc{GetLearnings}(t)$ **do**
    $\text{score}(\ell) \leftarrow s_t \times W[\ell.\text{valid\_level}]$
    $\mathcal{L}_{\text{pool}} \leftarrow \mathcal{L}_{\text{pool}} \cup \{(\ell, \text{score}(\ell))\}$
  end for
end for
Sort $\mathcal{L}_{\text{pool}}$ by score descending; deduplicate by learning text
$C \leftarrow$ first N_{pick} elements of $\mathcal{L}_{\text{pool}}$
return C

Algorithm 2 Hybrid Tool Retrieval

Require: Issue description I_q; knowledge base $\mathcal{K}$; result count k=3; dense weight α=0.7; sparse weight β=0.3; validation weights W

Ensure: Top-k issue–learning pairs

$q' \leftarrow \textsc{Clean}(I_q)$ {remove digits, normalize}
$\mathbf{v}_q \leftarrow \textsc{Embed}(q')$
$\mathbf{s}_{\cos} \leftarrow \textsc{CosineSim}(\mathbf{v}_q, \mathcal{K}.\text{issue\_embeddings})$
$\mathbf{s}_{\text{bm25}} \leftarrow \text{BM25}(q', \mathcal{K}.\text{index})$
$\mathcal{C} \leftarrow \text{Top}(\mathbf{s}_{\cos}, 100) \cup \text{Top}(\mathbf{s}_{\text{bm25}}, 100)$ {candidate pool}
for each $i \in \mathcal{C}$ **do**
  $c_i \leftarrow \text{clamp}(\mathbf{s}_{\cos}[i], 0, 1)$
  $b_i \leftarrow \textsc{MinMaxNorm}(\mathbf{s}_{\text{bm25}}[i], \mathcal{C})$
  $S_{\text{hybrid}} \leftarrow \alpha \cdot c_i + \beta \cdot b_i$
  $\text{score}(i) \leftarrow S_{\text{hybrid}} \times W[\mathcal{K}[i].\text{valid\_level}]$
end for
Sort $\mathcal{C}$ by score descending
return first k elements of $\mathcal{C}$

- **Sparse retrieval**: A BM25 index (Robertson and Zaragoza, 2009) is constructed over concatenated document fields (`obj_type`, `verbs`, `issue_text`, `learning_text`). BM25 scoring captures exact lexical matches that embedding similarity may miss.

Candidate selection and normalization. The candidate pool is the union of the top-100 results from each retrieval strategy. Scores are normalized to $[0, 1]$: cosine similarities are clamped, and BM25 scores are min-max normalized within the candidate set:

$$\text{BM25}_{\text{norm}}(i) = \frac{\text{BM25}(i) - \text{BM25}_{\min}}{\text{BM25}_{\max} - \text{BM25}_{\min}} \tag{4}$$

Score fusion and ranking. The normalized scores are combined via a weighted sum that prioritizes semantic similarity, then modulated by validation confidence:

$$S_{\text{hybrid}}(e) = 0.7 \times \cos_{\text{norm}}(e) + 0.3 \times \text{BM25}_{\text{norm}}(e) \tag{5}$$

$$\text{Score}_{\text{TR}}(e) = S_{\text{hybrid}}(e) \times \omega_e \tag{6}$$

where ω_e is the validation weight of entry e. Candidates are ranked by Score_{TR} and the top-k (k=3) results are returned to the agent as issue–solution pairs.

The full procedure is given in Algorithm 2.

3.3.4 Hard Negative Control

To verify that performance gains from CR and TR stem from the *relevance* of retrieved content rather than merely the presence of additional context in the prompt, we introduce a hard negative control. This ablation uses an identical retrieval pipeline but inverts the selection criterion: CR retrieves the *least* similar tasks (bottom-k by cosine similarity) and ranks learnings by score in *ascending* order, while TR selects candidates from the bottom-100 of each retrieval strategy. The result is a prompt augmented with the same number of learnings in the same format, but drawn from maximally irrelevant tasks. If the memory system's benefit derives from relevant content, the hard negative variant should degrade performance; if it merely provides useful prompt scaffolding, performance should be comparable.

3.4 Reflexion Baseline

As an upper-bound baseline, we compare against Reflexion (Shinn et al., 2023), an iterative self-correction framework. After each failed trial, the LLM generates a natural-language reflection analyzing what went wrong and proposing a revised plan. Subsequent trials include all prior reflections in context, allowing the agent to iteratively refine its strategy. We allow up to 7 trials per task.

Reflexion differs from our memory system in a fundamental way: it is *intra-task*—the agent retries the same task with accumulated self-critiques—whereas CR and TR are *cross-task*—learnings from other tasks are applied to new, unseen tasks. This distinction is important because Reflexion's gains come at substantially higher computational cost (cumulative steps across all trials), while our memory mechanisms operate in a single trial. We report both per-trial and per-task step counts to capture this difference.

4 Experimental Setup

4.1 Tasks and Splits

All agent reasoning is performed by Gemini 2.5 Flash (`gemini-2.5-flash`), and all retrieval embeddings are computed using Google's `text-embedding-004` model. These choices are held constant across all experiments to ensure that observed differences arise from the memory mechanisms rather than model variation.

ALFWorld. The full ALFWorld dataset (Shridhar et al., 2021) comprises 3,827 games split into a training set (3,553 games), a validation-seen set (140 games), and a validation-unseen set (134 games). For knowledge base construction, we use *alfworld-mini*, a curated subset of 120 training games (20 per task type × 6 task types). We run Reflexion on this subset and extract approximately 400 issue–learning entries from the resulting trajectories using the pipeline described in Section 3.3.1. The validation splits are identical to those of the full ALFWorld dataset, ensuring direct comparability with prior work. No training games appear in the validation sets.

WebShop. We partition the WebShop environment (Yao et al., 2022) into a development split (200 tasks) and a held-out test split (200 tasks). The knowledge base is extracted from Reflexion training trajectories on a separate set of training tasks with no overlap with either evaluation split.

Framework variants. We evaluate six framework variants in each environment:

1. **ReAct**: the base agent with no memory (Yao et al., 2023), serving as the primary baseline.
2. **ReAct + Reflexion**: the iterative self-correction baseline (Shinn et al., 2023), allowed up to 7 trials per task.
3. **ReAct + CR**: base agent augmented with Context Retrieval only.
4. **ReAct + TR**: base agent augmented with Tool Retrieval only.
5. **ReAct + CR + TR**: base agent augmented with both retrieval mechanisms.
6. **ReAct + hard-neg CR + TR**: ablation control that retrieves maximally *irrelevant* entries via inverted retrieval (Section 3.3.4).

All memory-augmented variants (CR, TR, CR+TR) execute a single trial per task, whereas Reflexion may execute up to 7 trials.

4.2 Evaluation Metrics

We report four metrics designed to capture both task performance and computational cost.

Accuracy (success rate). The fraction of tasks completed successfully, computed as a binary outcome per task: the agent either achieves the task goal or it does not. This is the primary metric for comparing framework effectiveness.

Average Steps per Trial. The mean number of thought–action–observation steps per individual trial attempt. For single-trial methods (ReAct, CR, TR, CR+TR, and the hard-negative control), each task involves exactly one trial. For Reflexion, this metric reflects the average length of a single trial attempt, irrespective of how many trials were needed.

Average Steps per Task. The mean total number of steps expended per task across all trials. For single-trial methods, this is identical to Steps/Trial. For Reflexion, this metric accumulates steps across all retry trials (up to 7), capturing the full computational cost of iterative self-correction. This distinction is critical: a method that matches Reflexion's accuracy at a fraction of its Steps/Task achieves a more favorable accuracy–cost tradeoff.

Gain from ReAct. The percentage-point difference between a given variant's accuracy and the ReAct baseline accuracy on the same evaluation split. This metric provides a normalized view of how much each mechanism contributes beyond the base agent's capability.

5 Results

We present results for all six framework variants across both environments. Tables 1–4 report accuracy, step counts, success ratios, and gain from the ReAct baseline. The central finding is stark: the same memory system produces substantial gains in ALFWorld and substantial losses in WebShop.

5.1 ALFWorld: Memory Helps

On ALFWorld, episodic memory consistently improves over the ReAct baseline across both validation splits.

Context Retrieval drives the largest single-mechanism gains. CR alone raises accuracy from 61.43% to 77.14% on seen tasks (+15.7pp) and from 61.94% to 76.12% on unseen tasks (+14.2pp). These gains are remarkably consistent across splits, suggesting that the knowledge base captures generalizable patterns rather than memorized task-specific solutions. Notably, CR achieves these improvements while *reducing* the average number of steps per trial relative to the baseline (23.09 vs. 24.29 on seen tasks). This indicates that proactively injecting relevant learnings does not merely add successful trajectories—it makes the agent more efficient by steering it away from common mistakes early in execution.

Tool Retrieval provides complementary gains, particularly on unseen tasks. TR alone yields a modest +2.1pp improvement on seen tasks (63.57%) but a more substantial +8.2pp on unseen tasks (70.15%). This asymmetry is informative: on-demand retrieval is most valuable when the agent encounters genuinely novel situations where proactive context may be insufficient. The stronger unseen-split gain suggests that TR's issue-level matching (Section 3.3.3) captures fine-grained obstacle patterns that generalize even when task-level similarity is lower.

Combining CR and TR yields the best memory-based performance. The CR+TR variant achieves 77.86% on seen tasks (+16.4pp) and 81.34% on unseen tasks (+19.4pp), the highest accuracy among all single-trial methods. On the unseen split, this nearly matches Reflexion's 83.58% (+21.6pp)—a gap of only 2.2pp—while requiring roughly one-third the computational budget: 23.35 steps per task for CR+TR versus 74.90 for Reflexion. This result demonstrates that cross-task memory transfer can approach the performance of intra-task iterative refinement (Shinn et al., 2023) at dramatically lower cost, provided that the environment supports meaningful transfer between tasks.

The hard-negative control confirms that retrieval relevance matters. Replacing relevant retrievals with maximally irrelevant ones degrades performance below the no-memory baseline: 55.71% on seen (−5.7pp) and 44.78% on unseen (−17.2pp). The more severe degradation on unseen tasks is consistent with the hypothesis that agents rely more heavily on retrieved context when facing unfamiliar situations. This ablation rules out the possibility that memory gains stem from prompt-length effects or generic scaffolding; the content of retrieved experiences, not their mere presence, drives improvement.

5.2 WebShop: Memory Hurts

On WebShop, the same memory system that improves ALFWorld performance causes severe degradation across all retrieval variants.

All memory variants underperform the baseline by wide margins. The ReAct baseline achieves 56.00% on the development split. CR reduces this to 27.00% (−29.0pp), TR to 34.50% (−21.5pp), and CR+TR to 25.50% (−30.5pp). Rather than helping, retrieved learnings actively mislead the agent. The pattern is monotonic: adding more retrieved context produces worse outcomes, with CR+TR performing worse than either mecha-

Table 1: ALFWorld validation-seen results (140 tasks). CR = Context Retrieval, TR = Tool Retrieval. **Bold** = best accuracy.

Framework	Success / Total	Accuracy	Steps/Trial	Steps/Task	Gain
ReAct	86 / 140	61.43	24.29	24.29	—
ReAct + Reflexion†	115 / 140	**82.14**	31.72	79.06	+20.71
ReAct + CR	108 / 140	77.14	23.09	23.09	+15.71
ReAct + TR	89 / 140	63.57	26.90	26.90	+2.14
ReAct + CR + TR	109 / 140	77.86	25.41	25.41	+16.43
ReAct + hard-neg CR + TR*	78 / 140	55.71	28.41	28.41	−5.71

†Max 7 trials; Steps/Task reflects cumulative cost. *Ablation control retrieving *least*-similar entries.

Table 2: ALFWorld validation-unseen results (134 tasks). CR = Context Retrieval, TR = Tool Retrieval. **Bold** = best accuracy.

Framework	Success / Total	Accuracy	Steps/Trial	Steps/Task	Gain
ReAct	83 / 134	61.94	22.38	22.38	—
ReAct + Reflexion†	112 / 134	**83.58**	30.41	74.90	+21.64
ReAct + CR	102 / 134	76.12	22.55	22.55	+14.18
ReAct + TR	94 / 134	70.15	24.93	24.93	+8.21
ReAct + CR + TR	109 / 134	81.34	23.35	23.35	+19.40
ReAct + hard-neg CR + TR*	60 / 134	44.78	33.01	33.01	−17.16

†Max 7 trials; Steps/Task reflects cumulative cost. *Ablation control retrieving *least*-similar entries.

nism alone. This suggests that retrieved learnings interfere with the agent's ability to navigate the environment effectively, likely by introducing spurious heuristics drawn from structurally dissimilar tasks.

The hard-negative control reveals a paradox of relevant retrieval. The hard-negative variant—which retrieves maximally *irrelevant* experiences—achieves 43.50% on the development split and 35.50% on the test split. Crucially, this *outperforms* all properly configured retrieval variants: CR (27.00%/31.50%), TR (34.50%/34.00%), and CR+TR (25.50%/23.00%). In ALFWorld, the hard-negative control degrades performance as expected (55.71% seen, 44.78% unseen—well below the correct CR+TR). On WebShop, the inversion is striking: irrelevant memories cause a modest −12.5pp degradation while "relevant" ones cause −30.5pp. We hypothesize that obviously irrelevant retrievals are easier for the agent to recognize and ignore, whereas semantically similar but non-transferable experiences are treated as trustworthy guidance and followed more confidently, producing worse outcomes. This finding suggests that in low-transfer environments, the danger is not noise but *active misdirection* by superficially plausible memories.

Only Reflexion improves over the baseline, but at high cost. Reflexion achieves 67.00% on dev (+11.0pp) and 60.00% on test (+9.50pp), the only variant to surpass the ReAct baseline on either split. However, this comes at 3.5–3.8× the step cost (85.13/107.38 vs. 24.44/27.97 steps per task). Reflexion's success here is consistent with its intra-task nature: rather than transferring learnings from other tasks, it iteratively refines strategy on the *same* task through self-generated critiques. This circumvents the transfer problem entirely, because the reflections are always relevant to the task at hand.

Test-set results confirm all findings. The complete test-split results (Table 4) corroborate every development-set finding. ReAct achieves 50.50%, CR drops to 31.50% (−19.0pp), TR to 34.00% (−16.5pp), and CR+TR to 23.00% (−27.5pp). Reflexion improves to 60.00% (+9.5pp) at 3.8× the step

Table 3: WebShop development set results (200 tasks). CR = Context Retrieval, TR = Tool Retrieval. **Bold** = best accuracy.

Framework	Success / Total	Accuracy	Steps/Trial	Steps/Task	Gain
ReAct	112 / 200	56.00	24.44	24.44	—
ReAct + Reflexion†	134 / 200	**67.00**	24.64	85.13	+11.00
ReAct + CR	54 / 200	27.00	29.39	29.39	−29.00
ReAct + TR	69 / 200	34.50	23.51	23.51	−21.50
ReAct + CR + TR	51 / 200	25.50	30.52	30.52	−30.50
ReAct + hard-neg CR + TR*	87 / 200	43.50	28.66	28.66	−12.50

†Max 7 trials; Steps/Task reflects cumulative cost across all trials. *Ablation control retrieving *least*-similar entries.

Table 4: WebShop test set results (200 tasks). CR = Context Retrieval, TR = Tool Retrieval. **Bold** = best accuracy.

Framework	Success / Total	Accuracy	Steps/Trial	Steps/Task	Gain
ReAct	101 / 200	50.50	27.97	27.97	—
ReAct + Reflexion†	120 / 200	**60.00**	26.81	107.38	+9.50
ReAct + CR	63 / 200	31.50	29.62	29.62	−19.00
ReAct + TR	68 / 200	34.00	25.92	25.92	−16.50
ReAct + CR + TR	46 / 200	23.00	31.36	31.36	−27.50
ReAct + hard-neg CR + TR*	71 / 200	35.50	31.43	31.43	−15.00

†Max 7 trials; Steps/Task reflects cumulative cost across all trials. *Ablation control retrieving *least*-similar entries.

cost. The hard-negative paradox also replicates: the hard-neg variant (35.50%) again outperforms both CR+TR (23.00%) and CR (31.50%). The consistency of these patterns across two independent 200-task splits confirms that the observed degradation—and the counterintuitive advantage of irrelevant over "relevant" retrieval—are robust phenomena rather than split-specific artifacts.

Why does retrieval hurt? We hypothesize that retrieved learnings in WebShop are *superficially similar but structurally non-transferable.* Because each WebShop task targets a unique product with unique attributes, a learning extracted from one product search (e.g., "filter by brand X to find shirts under $30") may match a new query by surface-level keywords but prescribe actions that are irrelevant or counterproductive for the actual target product. The embedding model captures semantic similarity between task descriptions, but semantic similarity does not entail procedural transferability in environments with high instance variability.

5.3 Cross-Environment Comparison

The most striking result of this study is the magnitude and direction of the divergence: the same memory system, the same base LLM (Gemini 2.5 Flash), and the same knowledge base construction pipeline produce a ∼47pp swing in effect size across the two environments. On the held-out splits, CR+TR yields +19.4pp on ALFWorld unseen versus −27.5pp on WebShop test—a net difference of 46.9 percentage points. The development splits show a nearly identical gap: +16.4pp (ALFWorld seen) versus −30.5pp (WebShop dev), a 46.9pp swing. The consistency of this ∼47pp divergence across independent split pairs confirms that it is attributable to environment properties rather than evaluation-set idiosyncrasies.

Retrieval relevance helps in high-transfer environments but backfires in low-transfer ones. The hard-negative controls reveal an asymmetry between environments. On ALFWorld, inverting retrieval degrades accuracy by 22–37pp relative to the correct CR+TR variant, confirming that relevant content matters when transfer is possible. On Web-

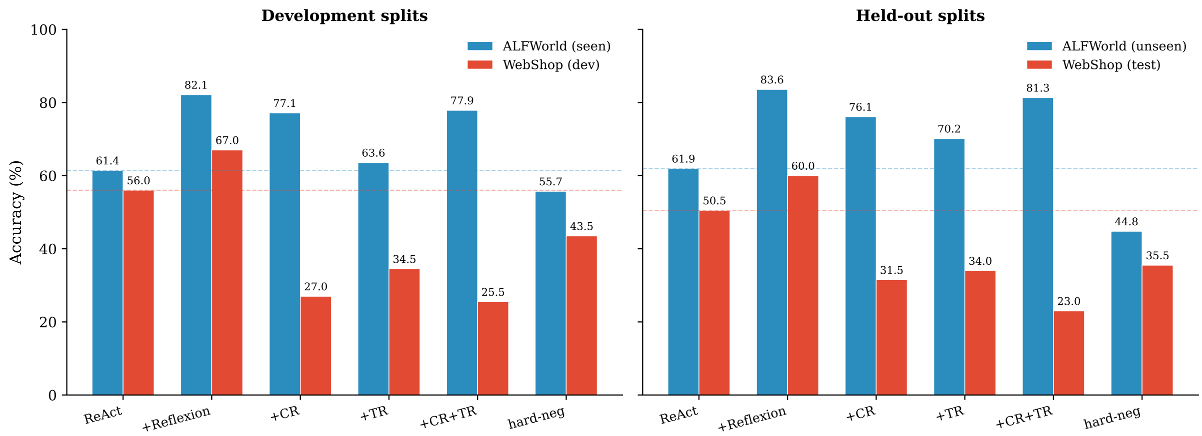

Figure 1: Accuracy comparison across all framework variants, with properly paired splits: development (ALFWorld seen vs. WebShop dev, left) and held-out (ALFWorld unseen vs. WebShop test, right). The same memory mechanisms improve ALFWorld performance but degrade WebShop performance across both pairings. Dashed lines indicate ReAct baselines.

Shop, the pattern inverts: the hard-negative control (43.50% dev, 35.50% test) *outperforms* the correctly configured CR+TR (25.50% dev, 23.00% test) by 12–18pp. This paradox indicates that semantic similarity between task descriptions is not merely an imperfect proxy for transferability—it is actively counterproductive in low-transfer environments, because it selects experiences that are superficially plausible but procedurally misleading.

Reflexion succeeds in both environments because it sidesteps transfer. Reflexion improves over the baseline by +20.7pp (seen) and +21.6pp (unseen) on ALFWorld, and by +11.0pp/+9.5pp (dev/test) on WebShop. Its consistent gains across environments of very different character underscore that intra-task self-correction does not depend on cross-task transfer at all—it only requires that the LLM can generate useful self-critiques for the specific task it is retrying. The trade-off is computational: Reflexion's cumulative step cost is 3–4× that of single-trial methods, making cross-task memory the more efficient option *when the environment supports it.*

Implications for practitioners. Figure 1 visualizes the divergence across properly paired splits (development and held-out), and Figure 2 shows the gain from baseline for each variant across all four evaluation splits.

These results suggest that episodic memory augmentation should not be applied indiscriminately. Before investing in knowledge base construction and retrieval infrastructure, practitioners should assess whether their target environment exhibits the properties that enable cross-task transfer: compositional task structure, a regular action space, and sufficient overlap between tasks such that individual learnings have a broad transfer radius. When these conditions hold, as in ALFWorld, even simple embedding-based retrieval can approach the accuracy of iterative self-correction at a fraction of the cost. When they do not, as in WebShop, the same retrieval mechanisms actively degrade performance, and intra-task methods like Reflexion remain the safer choice despite their higher computational overhead. We formalize this diagnostic framework in Section 6.

6 Analysis and Discussion

The preceding results reveal a striking divergence: the same episodic memory system yields substantial improvements on ALFWorld yet actively degrades performance on WebShop. In this section, we trace this divergence to three structural properties of the task environment and synthesize them into a diagnostic framework for practitioners.

6.1 Task Compositionality

ALFWorld tasks are inherently compositional. Each of the six task types decomposes into a canonical sequence of sub-goal phases—SEARCH → ACQUIRE → TRANSFORM → PLACE—and these phases are shared across

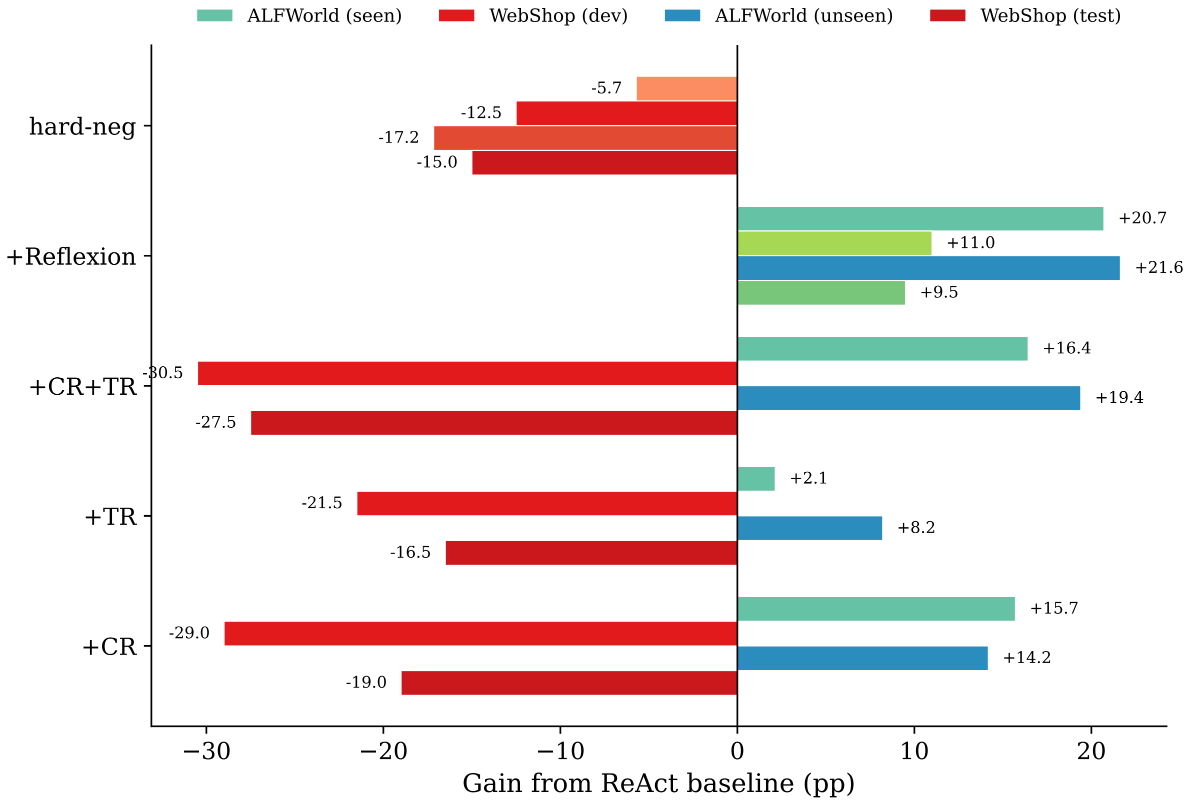

Figure 2: Gain from ReAct baseline (percentage points) for each memory variant across all four evaluation splits. Green/blue indicates improvement; orange/red indicates degradation. Development splits (ALFWorld seen, WebShop dev) and held-out splits (ALFWorld unseen, WebShop test) show consistent patterns, confirming that the ∼47pp swing between environments is robust.

task types. A learning acquired during the Search phase of a *pick_and_place* task (e.g., "check common receptacle locations for mugs") applies equally to the Search phase of a *pick_heat_then_place* or *pick_cool_then_place* task, because the sub-goal structure is the same: the agent must locate an object regardless of what it will subsequently do with it. This structural overlap means that the knowledge base need only capture a modest set of phase-specific learnings to cover a combinatorially large space of concrete tasks.

WebShop tasks, by contrast, are monolithic. Each task requires the agent to locate a unique product through a unique path in the catalog—there is no shared sub-task decomposition analogous to ALFWorld's goal phases. Searching for a "red cotton shirt under \$30" and searching for a "stainless steel water bottle with straw lid" share no meaningful sub-goal structure: the queries differ, the product pages differ, and the attribute-matching criteria differ. Consequently, a learning extracted from one product search rarely generalizes to another.

This distinction is reflected directly in our results. Context Retrieval produces a +15.7pp improvement on ALFWorld seen tasks, where compositional sub-goals enable retrieved learnings to transfer across tasks that share goal phases. On WebShop, the same mechanism yields a −29pp degradation, because retrieved learnings from structurally unrelated product searches introduce irrelevant guidance that misleads the agent. The hard-negative paradox reinforces this interpretation: in WebShop, obviously irrelevant retrievals (−12.5pp) cause less harm than semantically "relevant" ones (−30.5pp), because the agent can more easily discard guidance that bears no surface resemblance to the current task, whereas plausible-looking but non-transferable memories are followed with misplaced confidence.

Principle. Episodic memory is most effective when tasks share compositional sub-task structure, enabling learnings about individual phases to generalize across task instances.

6.2 Action Space Regularity

ALFWorld operates with a fixed action grammar of approximately 15 verb templates (`go to`, `take`, `put`, `open`, `close`, `heat`, `cool`, `clean`, `use`, `examine`, etc.). This constrained vocabulary means that learnings about action sequences—such as "open a container before attempting to take an object from it"—are expressed in the same terms the agent will use in future tasks. The regularity of the action space ensures that procedural knowledge, once acquired, remains applicable: there is only one way to open a microwave (`open microwave 1`), and this action template is identical across all tasks that involve microwaves.

WebShop's action space is syntactically simple—`search[query]` and `click[element]`—but combinatorially vast. The `search` action accepts free-form text, meaning that the effective action space is the set of all possible English queries. The `click` action targets dynamically generated page elements that vary across products and search results. Even two tasks that are superficially similar (both searching for shirts) require entirely different search queries and click sequences, because the specific products, attributes, and page layouts differ. Learned action sequences are therefore tied to specific navigation paths that do not recur.

Our results corroborate this analysis. Tool Retrieval, which provides action-level guidance during execution, produces gains on ALFWorld (+2.1pp seen, +8.2pp unseen) but degrades performance on WebShop (−21.5pp). In ALFWorld's regular action space, mid-execution advice about action sequencing generalizes; in WebShop's irregular space, action-specific guidance from a different product search is worse than no guidance at all.

Principle. Episodic memory is most effective when the action space is constrained and regular, so that learned action sequences remain applicable across tasks.

6.3 Learning Transfer Radius

We define the *learning transfer radius* as the number of distinct future tasks that benefit from a single learned experience. This property captures the expected return on investment from storing any given learning in the knowledge base.

In ALFWorld, the transfer radius is high. A single learning such as "open the container before taking an object" applies to dozens of tasks across multiple task types—any task involving object retrieval from a closed receptacle. The knowledge base contains approximately 400 entries covering 6 task types with roughly 50 object types, yet each learning potentially benefits 5–10 or more distinct task instances. This high transfer radius means that even a modestly sized knowledge base achieves broad coverage.

In WebShop, the transfer radius approaches zero. A learning about how to navigate to a specific product (e.g., "search for 'organic green tea bags 100 count' and select the third result") is tied to that unique product and is functionally useless for any other task. The knowledge base may contain hundreds of entries, but each entry addresses a singular, non-recurring situation.

The empirical signature of transfer radius is visible in the gap between seen and unseen performance. On ALFWorld, the CR+TR configuration achieves +19.4pp on unseen tasks, *exceeding* its +16.4pp gain on seen tasks. This indicates that learnings extracted from training tasks transfer effectively to novel task configurations—precisely what high transfer radius predicts. On WebShop, no such generalization occurs; performance degrades regardless of whether the test tasks were seen during knowledge base construction. The hard-negative results provide further evidence: in a near-zero-transfer environment, even the *direction* of retrieval relevance inverts—irrelevant memories (43.50%) outperform "relevant" ones (25.50%)—because with no transferable knowledge to leverage, semantic similarity selects precisely those entries most likely to produce confident but wrong guidance.

Principle. Episodic memory is most effective when the learning transfer radius is high—that is, when individual learnings generalize to many future tasks rather than being tied to unique, non-recurring situations.

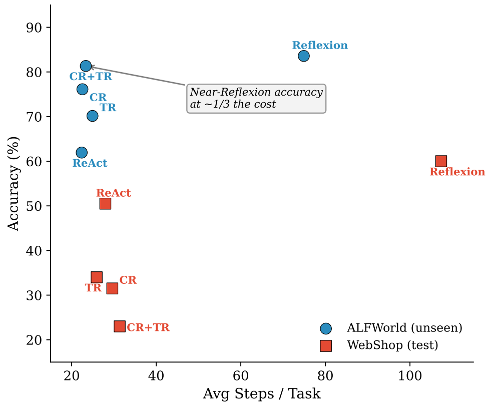

Figure 3: Accuracy vs. computational cost (average steps per task) on held-out splits: ALFWorld unseen (circles) and WebShop test (squares). On ALFWorld, CR+TR achieves near-Reflexion accuracy at ∼1/3 the step cost. On WebShop, memory variants degrade accuracy without reducing cost. Reflexion's high step cost on WebShop test (107 steps/task) reflects up to 7 retry trials.

Table 5: Diagnostic framework for predicting episodic memory effectiveness. The three structural properties consistently distinguish environments where memory helps (ALFWorld) from those where it hurts (WebShop). Effect sizes are relative to the ReAct baseline; ALFWorld values are seen/unseen, WebShop values are dev/test.

Property	ALFWorld	WebShop
Task Compositionality	High (shared sub-goals)	Low (unique products)
Action Space Regularity	High (fixed, ∼15 verbs)	Low (free-form)
Learning Transfer Radius	High (1 → many tasks)	Near-zero
Memory Effect (CR+TR)	+16.4/+19.4pp	−30.5/−27.5pp
Hard-neg Effect*	−5.7/−17.2pp	−12.5/−15.0pp

*Hard-neg *outperforms* proper retrieval on WebShop (−12.5 vs. −30.5pp), indicating that semantic similarity is counterproductive when transfer radius is near zero.

6.4 When Does Memory Work? A Diagnostic Framework

The three properties identified above—task compositionality, action space regularity, and learning transfer radius—together predict whether episodic memory augmentation will benefit an agent. Table 5 summarizes how the two environments in our study differ along these dimensions and the corresponding empirical outcomes.

The pattern is clear: when all three properties favor transfer, episodic memory provides substantial and consistent gains; when they do not, retrieved experiences actively mislead the agent by introducing irrelevant procedural guidance that competes with the agent's own reasoning.

This framework yields a practical recommendation for practitioners considering episodic memory augmentation. Before deploying a memory system, one should assess the target environment along these three dimensions:

1. **Task compositionality**: Do tasks decompose into shared sub-goals, or is each task a unique, monolithic sequence?

2. **Action space regularity**: Is the action vocabulary constrained and reusable, or is it open-ended with combinatorially many possibilities?

3. **Learning transfer radius**: Will a typical learning apply to many future tasks, or is it tied to a specific, non-recurring situation?

If all three properties favor transfer, episodic memory is likely beneficial. If any property is unfavorable, practitioners should proceed with caution—our WebShop results demonstrate that memory can degrade performance substantially when the environment's structure does not support learning transfer. In such cases, alternative strategies—such as in-context learning without persistent memory, or memory systems with explicit gating mechanisms that suppress retrieval when confidence is low—may be more appropriate.

7 Limitations

Our study has several limitations that should be considered when interpreting the results and applying the diagnostic framework.

Single base LLM. All experiments use Gemini 2.5 Flash as the base language model. Different LLMs may exhibit different sensitivities to retrieved context—models with stronger instruction-following capabilities might better distinguish relevant from irrelevant retrieved learnings, potentially mitigating the degradation observed on WebShop. Conversely, weaker models might fail to leverage even high-quality retrievals. The interaction between memory augmentation and base model capability remains an open question.

Two environments. Our diagnostic framework is derived from observations across two environments. While ALFWorld and WebShop represent meaningfully different points in the space of task properties, validation on additional domains—such as ScienceWorld (Wang et al., 2022), Minecraft (Fan et al., 2022), or web navigation benchmarks like MiniWoB++ (Liu et al., 2018)—would strengthen confidence in the framework's generality. Environments with intermediate levels of compositionality or action space regularity would be particularly informative.

Knowledge base provenance. The knowledge base is constructed from training-split trajectories within each environment. Although the strong unseen-task results on ALFWorld (+19.4pp for CR+TR) argue against simple overfitting to the training distribution, we cannot rule out that the knowledge base captures environment-specific regularities that would not emerge from a more diverse training corpus.

No hyperparameter ablations. We do not ablate over knowledge base size, retrieval depth k, or the dynamic quota parameters governing context window allocation. It is possible that different hyperparameter settings could improve WebShop performance or further improve ALFWorld performance. The sensitivity of episodic memory systems to these choices warrants systematic investigation.

Hard-negative paradox interpretation. The finding that irrelevant memories outperform "relevant" ones in WebShop (43.50% vs. 25.50% on dev, 35.50% vs. 23.00% on test) is consistent across both splits, but the precise mechanism—whether the agent ignores obviously irrelevant content, or whether semantically similar content triggers overconfident adherence—warrants further investigation through trajectory-level analysis or attention probing.

Fixed embedding model. All retrieval experiments use a single embedding model (Google's `text-embedding-004`). Alternative embedding models—particularly those fine-tuned for task-specific similarity or trained on agent trajectory data—might yield different retrieval quality and consequently different downstream performance.

8 Conclusion

This work demonstrates that episodic memory is not a universal enhancement for LLM-based agents. Through a controlled study applying an identical memory system across two interactive environments, we show that the same retrieval mechanisms produce opposite outcomes: up to +19.4 percentage points improvement on ALFWorld versus up to −30.5pp degradation on WebShop. This ∼47pp swing, consistent across development and held-out splits and driven entirely by environment properties rather than system design choices, challenges the prevailing assumption that storing and retrieving past experience is inherently beneficial.

Our analysis traces this divergence to three structural properties: task compositionality, action space regularity, and learning transfer radius. When tasks decompose into shared subgoals, the action vocabulary is constrained and reusable, and individual learnings generalize to many future tasks, episodic memory provides substantial gains—in ALFWorld's case, at roughly one-third the step cost of Reflexion. When these properties are absent, as in WebShop, retrieved experiences introduce misleading procedural guidance that actively degrades performance. A striking corollary emerges from our hard-negative ablation: in low-transfer environments, irrelevant memories cause *less* harm than semantically "relevant" ones, suggesting that the failure mode is not noise injection but confident misdirection by superficially plausible but non-transferable experiences.

Beyond the empirical findings, we contribute a diagnostic framework that gives practitioners actionable criteria for deciding whether episodic memory augmentation is appropriate for a given deployment setting. Rather than presenting another memory-augmented agent that claims universal improvement, we characterize the conditions under which memory helps and, critically, the conditions under which it hurts.

Several directions for future work emerge from this study. First, *adaptive memory gating*—mechanisms that dynamically detect when retrieved experiences are likely to be helpful and suppress retrieval otherwise—could enable memory systems to operate safely across diverse environments. Second, validation on additional domains with varying structural properties (e.g., ScienceWorld, Minecraft, web navigation) would test the generality of our diagnostic framework. Third, *memory filtering* strategies that assess transferability before injecting retrieved content could mitigate degradation in low-compositionality settings. Finally, a scaling analysis examining whether larger knowledge bases or more diverse training data can overcome the transfer limitations observed in WebShop would clarify the boundary between environment structure and data sufficiency as determinants of memory effectiveness.

References

Linxi Fan, Guanzhi Wang, Yunfan Jiang, Ajay Mandlekar, Yuncong Yang, Haoyi Zhu, Andrew Tang, De-An Huang, Yuke Zhu, and Anima Anandkumar. 2022. MineDojo: Building open-ended embodied agents with internet-scale knowledge. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Patrick Lewis, Ethan Perez, Aleksandara Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Evan Zheran Liu, Kelvin Guu, Panupong Pasupat, Tianlin Shi, and Percy Liang. 2018. Reinforcement learning on web interfaces using workflow-guided exploration. In *International Conference on Learning Representations (ICLR)*.

Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. 2023. Self-refine: Iterative refinement with self-feedback. *Advances in Neural Information Processing Systems (NeurIPS)*.

Joon Sung Park, Joseph C O'Brien, Carrie J Cai, Meredith Ringel Morris, Percy Liang, and Michael S Bernstein. 2023. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology (UIST)*.

Stephen Robertson and Hugo Zaragoza. 2009. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389.

Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Mohit Shridhar, Xingdi Yuan, Marc-Alexandre Côté, Yonatan Bisk, Adam Trischler, and Matthew Hausknecht. 2021. ALFWorld: Aligning text and embodied environments for interactive learning. In *International Conference on Learning Representations (ICLR)*.

Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. 2023. Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv:2305.16291*.

Ruoyao Wang, Peter Jansen, Marc-Alexandre Côté, and Prithviraj Ammanabrolu. 2022. ScienceWorld: Is your agent smarter than a 5th grader? In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing (EMNLP)*.

Shunyu Yao, Howard Chen, John Yang, and Karthik Narasimhan. 2022. WebShop: Towards scalable real-world web interaction with grounded language agents. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*.

Wanjun Zhong, Lianghong Guo, Qiqi Gao, He Ye, and Yanlin Wang. 2024. MemoryBank: Enhancing large language models with long-term memory. *Proceedings of the AAAI Conference on Artificial Intelligence*.

Xizhou Zhu, Yuntao Chen, Hao Tian, Chenxin Tao, Weijie Su, Chenyu Yang, Gao Huang, Bin Li, Lewei Lu, Xiaogang Wang, Yu Qiao, Zhaoxiang Zhang, and Jifeng Dai. 2023. Ghost in the Minecraft: Generally capable agents for open-world environments via large language models with text-based knowledge and memory. *arXiv preprint arXiv:2305.17144*.